



WELCOME EVERYONE



TOPIC: ARRAYS, CLASSES, STRUCTURES, ENUMERATIONS

Presented by: Bishal Pandey
Roll NO.17

ARRAY

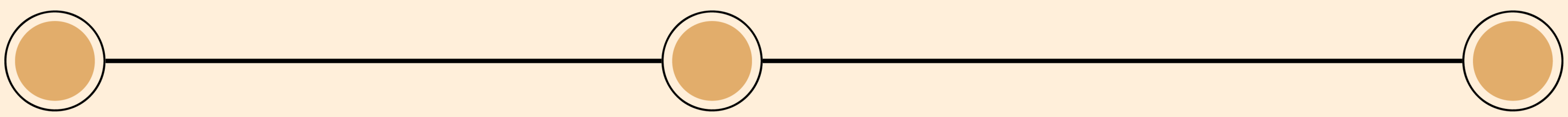


An array in C# is a container that holds a group of value of the same type, Stored in a specific order, and accessed using a index.

Syntax: `type[] arrayName = { value1, value2, value3 };`

- **`type[]` → defines the data type (like int, string, etc.)**
- **`arrayName` → the name of your array**
- **`{ ... }` → the values stored in the array**
- **`arrayName[index]` → accesses a specific item (starting from 0)**

FEATURE OF ARRAY IN C#



1. Indexed Access

You can instantly access any element using its index—like `array[0]` for the first item. This makes reading and writing data super fast.

2. Fixed Size

Once you define the size of an array, it stays constant. This helps with memory control and predictable structure.

3. Loop-Friendly

Arrays work perfectly with loops (for, foreach), making it easy to process or display all elements efficiently.

TYPE OF ARRAY

Single-Dimensional Array

1	2	3	4	5
[0]	[1]	[2]	[3]	[4]

Single-Dimensional

Multidimensional Array

[0]	1	2	3
[1]	4	5	6
	[0]	[1]	[2]

Multidimensional

Jagged Array

[0]	1	2	
[1]	3	4	5
	[0]	[1]	[2]

Jagged

Single-Dimensional Array

1	2	3	4	5
[0]	[1]	[2]	[3]	[4]

SINGLE-DIMENSIONAL ARRAY

A single-dimensional array is just a list of values stored in order. Think of it like a row of boxes, each holding one item.

```
using System;

class Program
{
    static void Main()
    {
        // Create an array with values 1 to 5
        int[] numbers = { 1, 2, 3, 4, 5 };

        // Print each value with its index
        Console.WriteLine("Array Elements:");
        for (int i = 0; i < numbers.Length; i++)
        {
            Console.WriteLine($"Index {i}: {numbers[i]}");
        }
    }
}
```

Output:

Array Elements:
Index 0: 1
Index 1: 2
Index 2: 3
Index 3: 4
Index 4: 5

Multidimensional Array

[0]	1	2	3
[1]	4	5	6
	[0]	[1]	[2]

MULTIDIMENSIONAL

A multidimensional array is like a table with rows and columns. It stores data in a grid format—more than one dimension.

```
using System;

class Program
{
    static void Main()
    {
        // Create a 2D array
        int[,] numbers = { {1, 2}, {3, 4} };

        // Print each value with its position
        Console.WriteLine("Multidimensional Array:");
        for (int row = 0; row < 2; row++)
        {
            for (int col = 0; col < 2; col++)
            {
                Console.WriteLine($"[{row},{col}] = {numbers[row, col]}");
            }
        }
    }
}
```

```
[0,0] = 1
[0,1] = 2
[1,0] = 3
[1,1] = 4
```

Jagged Array

[0]	<table><tr><td>1</td><td>2</td></tr></table>		1	2		
1	2					
[1]	<table><tr><td>3</td><td>4</td><td>5</td></tr></table>			3	4	5
3	4	5				
	[0]	[1]	[2]			

JAGGED ARRAY

A jagged array is an array of arrays, where each inner array can have a different length.

```
using System;

class Program
{
    static void Main()
    {
        // Create a jagged array with 2 rows
        int[][] jagged = new int[2][];

        // First row has 2 elements
        jagged[0] = new int[] { 1, 2 };

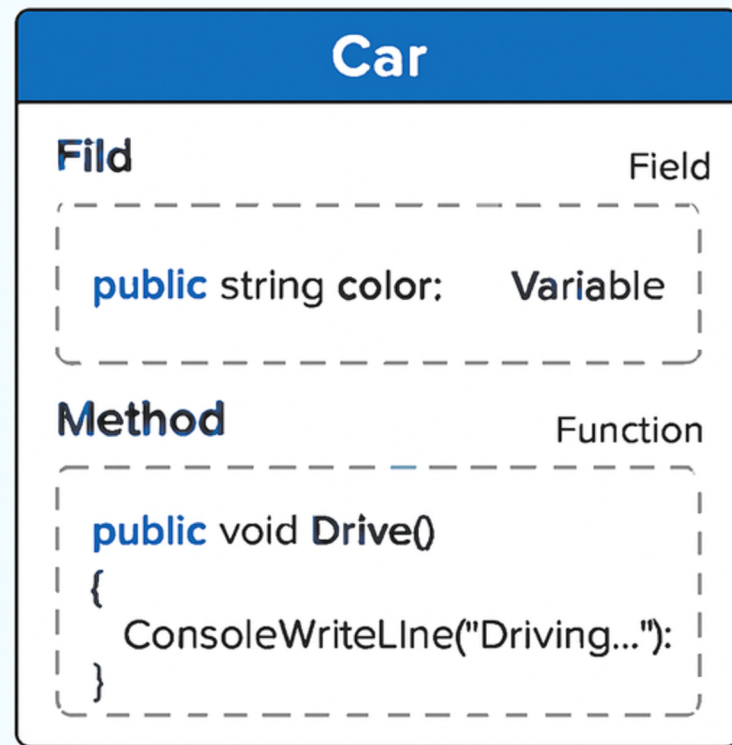
        // Second row has 3 elements
        jagged[1] = new int[] { 3, 4, 5 };

        // Print each element with its position
        Console.WriteLine("Jagged Array:");
        for (int i = 0; i < jagged.Length; i++)
        {
            for (int j = 0; j < jagged[i].Length; j++)
            {
                Console.WriteLine($"[{i}][{j}] = {jagged[i][j]}");
            }
        }
    }
}
```

```
[0][0] = 1
[0][1] = 2
[1][0] = 3
[1][1] = 4
[1][2] = 5
```


CLASS IN C#

A class in C# is a template that defines the structure and behavior of objects. It contains variables and function that describe what an object is and what it can do.



Class in C# = Contains Fields (Variables) and Methods (Functions)

```
class Person
{
    public string name;
    public void SayHello()
    {
        Console.WriteLine("Hello!");
    }
}
```

```
Person p = new Person();
p.name = "Bishal";
p.SayHello(); // Output: Hello!
```

FEATURE OF CLASS IN C#

1 Encapsulation

- Groups related data and methods together in one unit.

2 Data Members (Fields)

- Stores values like numbers, strings, etc.

3 Member Functions (Methods)

- Performs actions or operations using the data.

4 Properties

- Special methods to get and set field values safely.

5 Constructors

- Special methods that run when an object is created.

6 Access Modifiers

- Controls visibility (e.g., public, private, protected).

7 Inheritance Support

- One class can inherit from another to reuse code.

STRUCTURE IN C#

A structure in C# is a value type that groups related variables under one name. It is used to create lightweight objects that hold data but don't need full object-oriented features like inheritance.

Key Points:

- Defined using the struct keyword
- Stored in stack memory (not heap like classes)
- Good for small data models like coordinates, colors, or settings
- Cannot inherit from another struct or class
- Can have fields, methods, properties, and constructors

```
struct Point
{
    public int X;
    public int Y;

    public void Display()
    {
        Console.WriteLine($"X = {X}, Y = {Y}");
    }
}

Point p;
p.X = 5;
p.Y = 10;
p.Display(); // Output: X = 5, Y = 10
```

ENUMERATION (ENUM) IN C#

An enumeration (or enum) is a special type in C# that lets you define a set of named constant values. It's used when you want to work with a fixed group of related options.

Simple Features of Enumeration (enum) in C#

-  Gives names to numbers Makes your code easier to read instead of using raw numbers.
-  Fixed set of values You define all possible options up front.
-  Starts from 0 by default First item is 0, next is 1, and so on– unless you set custom values.
-  Can assign custom numbers You can manually set values like Success = 1, Error = -1.
-  Type-safe Only valid enum values can be used– helps avoid mistakes.

```
using System;

class Program
{
    // Define an enum for traffic light colors
    enum TrafficLight
    {
        Red,
        Yellow,
        Green
    }

    static void Main()
    {
        // Set the current light
        TrafficLight current = TrafficLight.Green;

        // Print the name of the light
        Console.WriteLine("Current light: " + current);

        // Print the numeric value of the light
        Console.WriteLine("Numeric value: " + (int)current);
    }
}
```



Current light: Green
Numeric value: 2

THANK
YOU

